

The entire JVM Architecture is divided into 3 sections

- 1) Class Loader sub system
- 2) Runtime Data areas (Memory Areas)
- 3) Execution Engine

The main purpose of Class Loader sub system to load the required .class file into JVM Memory from different memory locations.



Background: Differential class leader

It has the highest priority because Bootstrap class loader is the super class for Platform loader.

It loads the classes from the following path
C:\Program Files\Java\jdk-1.6.0_10\jre\lib\rt.jar

Before you fix, this class loader is

C → Program Size → Java → X86 → lib → end (extension) → 3rd Party per file. (split)(4 per)

From Java™ 9 onwards we have a new `CONCEPT` called `Instance` of course, but this one can directly use `Module`.

Example : java.exe, java.compiler and so on.

Command to create the jar file :

If we want to compile more than one java source file at a time then the command is : `javac *.java`

Application/System dass hier :
 Es ist empfohlen zu lesen all records

It has the lowest priority because it is the task class Platform class inside

to create the .class file from class path and of environment variable.

Note :-

If all the class loaders are failed to load the class file into JVM memory then we will get a runtime exception i.e. `java.lang.ClassNotFoundException`.

Base Delegation Hierarchy algorithm works as:

The class loader sub system follows Delegation Hierarchy Algorithm.



Whenever JVM makes a request to class loader & OS system to load the required class file into JVM memory, first of all, class loader sub system makes a request to Application class loader, Application class loader will delegate to class the request to the Platform Class loader, Platform class loader will also delegate the request to Bootstrap class loader.

Bootstrap class loader will load the `Class` file from `bootstrap.jar` and then it will pass the request back to Platform class loader. Platform class loader will load the `Class` file from `platform.jar` (3rd party jar file).
 (4) JRE (Java Runtime Module System) will pass the request back to Application class loader. It will load the `Class` file from `application` directory into `APP` memory.

Note : `java.lang.Object` is the first class to be loaded into JVM Memory